Memory Operations in MCP Agent Ecosystems: an Operation Taxonomy, an Evaluation Protocol, and a Reference Implementation

Konstantin Parashchuk*

Version 1.0, 16 October 2026

Abstract

Agent memory has become a persistent store with its own lifecycle, while its evaluation is still dominated by end-to-end task success measured over a monolithic black box [60, 27]. Model Context Protocol (MCP) servers now expose memory to agents through a standard interface [15, 13], and MCP servers are evaluated at task level [33, 34], yet no shared, operation-level protocol lets a reader run two memory systems and compare the resulting report. This paper contributes four artefacts. (i) A taxonomy of six memory operations — WRITE, READ, CONSOLIDATE, REVISE, INTEND, ASSERT — defined by preconditions, postconditions and externally checkable evidence, together with two mandatory record axes (PROVENANCE, LIFECYCLE). (ii) An evaluation protocol with ten task families, metric definitions, a run procedure and a machine-validated report format. (iii) A reference implementation of five of the six operations as public MCP servers, with its gaps stated before any measurement. (iv) Two validated protocol reports of two memory servers, produced by that protocol in a store-only configuration: 10 task families, 3 repetitions per family at seeds 1, 2, 3, 52 defined metrics per report with percentile bootstrap confidence intervals, and per-operation profiles that are read side by side rather than ranked, in tables generated from the report files rather than typed. The pair separates the two servers on the write path (write precision 1.0 against 0.75, with 4.0 idempotency violations in one and 0.0 in the other) and reproduces the same readings where the mechanism is the same (restart survival, provenance correctness and abstention on out-of-coverage questions). It surfaces one operation-level difference that retrieval scores hide: both servers still serve superseded content after a revision call, one of them while reporting the revision itself as correct. What is *not* measured is stated with the results: token and cost metrics are null by construction because no model is in the loop, the sample is two servers from one author's reference set, and the confidence intervals rest on three repetitions. All 135 sources curated for this work — 61 of them cited in this paper, the remainder in the companion taxonomy, protocol and related-work documents — were resolved by machine queries against the arXiv API, Crossref and direct HTTP probes; the resolution table, a report validator with negative tests, and the generator of the comparison tables ship with the paper.

Keywords: agent memory, Model Context Protocol, belief revision, prospective memory, confidence calibration, evaluation protocol, reproducibility.

1 Introduction

Large language model agents are increasingly built by reorganising the runtime around the model rather than by changing its weights: state moves into memory stores, expertise into

*Independent researcher, Saint Petersburg, Russia. Author contact and citation metadata in `CITATION.cff`. AI assistance in preparing this manuscript (literature resolution, artefact collection, drafting) is disclosed in the Acknowledgements.

reusable skills, and interaction into protocols [59]. The consequence for memory is a change of kind, not of degree. Memory is no longer a retrieval aid bolted onto a context window; it is a data management system that supports persistent storage, retrieval, update, consolidation and lifecycle governance across an agent's execution [60]. Two recent surveys describe the field along memory substrate, cognitive mechanism and subject [27] and along memory sources, forms and operations [57].

Evaluation, however, has not followed. The dominant practice is to score memory through end-to-end task success while treating the memory system as a monolithic black box [60], and benchmark papers frequently omit the information a reader needs to reproduce a comparison at all: scaffold, prompts, budgets [35]. The gap matters most where memory servers are pluggable. MCP has become the dominant way to give agents external capability [10, 15], its exact revision is part of every run's configuration [13, 14], and MCP servers are already benchmarked — for tool use [33, 34, 24], for safety [61] — with documented sampling biases that make reported comparisons hard to interpret [16]. Memory servers enter that ecosystem with no shared evaluation vocabulary.

Contributions.

1. **C1 (taxonomy, Section 3).** Six memory operations defined by pre/postconditions and observable evidence, plus two record axes (PROVENANCE, LIFECYCLE), mapped onto seven existing systems. Distinguishing feature: operations are interface-level, so a system that lacks one is visible as a gap rather than as a slightly lower accuracy score.

2. **C2 (protocol, Section 4).** Ten task families, a metric table in which each metric names its computer (harness, programmatic check, or two-grader rubric), a run procedure with budgets and repetitions, and a report format with a mandatory disclosure block and a JSON-Schema validator.

3. **C3 (reference implementation, Section 5).** Five of the six operations implemented as public MCP servers, with commit-pinned facts and an explicit gap map listed before any measurement.

4. **C4 (measured profiles, Section 6).** Two protocol reports of two memory servers that pass the report validator, with per-family metrics, bootstrap confidence intervals, raw MCP stdio logs, and per-operation profiles written as a pair rather than as a ranking. Every table and every inline number in that section is generated from the two report files by one utility, and the check that regenerates them fails if they drift.

5. **C5 (integrity artefacts).** A machine-generated reference verification table (135 sources, 0 unresolved), a report validator with negative tests, and the table generator, all shipped in the repository.

What this paper does not claim. It does not rank the two servers it measures: Section 6 reports per-operation profiles with their confidence intervals, and the two subjects differ in storage design by construction. It does not claim that its reference implementation is better than the systems it cites: one of its servers now has a measured profile, the others do not. It does not claim that human-like forgetting improves agent memory — the opposite is reported for language models [49]. It does not claim that the taxonomy is complete, nor that the measured pair generalises beyond the two servers it contains. The full ledger of claims, with what would refute each, is in `claims.md`.

2 Background

Memory as a data management problem. The four-module decomposition proposed in [60] — representation and storage, extraction, retrieval and routing, maintenance — is the most useful recent framing, and its authors evaluate twelve systems across five workloads, finding that no single architecture dominates and that localised maintenance is more cost-efficient than global reorganisation. Our protocol accepts that finding as a hypothesis to be tested per operation: it is designed to produce a per-operation profile rather than a ranking.

Vocabulary borrowed from cognitive science, deliberately narrowly. We borrow the separation of declarative and procedural memory with activation-based retrieval [29, 17], the observation that store boundaries differ between architectures [30], prospective memory as a faculty distinct from retrospective recall [21], source monitoring as a separable process [28], encoding specificity [47], and offline consolidation [48]. The analogy is used to name operations and to choose what to measure. It is not used to claim that artificial memory should be built to decay like human memory: what evidence exists points the other way [49], so decay appears in the taxonomy as a policy variant to be measured.

Belief change. Revision operations are named after the AGM tradition [22] with its truth-maintenance lineage [23]; we do not claim that memory stores satisfy the postulates, only that a revision policy must be stated in advance if revision correctness is to be scored. Conflict classes are taken from [53, 44], and answer-level evaluation is known to hide evidence use [19, 41].

MCP. The protocol targets MCP tool calls: server and client discovery, tools, resources, prompts [13], framed by JSON-RPC 2.0 [11], with metadata integrity already a demonstrated concern [39].

3 Taxonomy of memory operations

An operation is defined by a precondition, a postcondition, observable evidence and a set of failure modes; it is not defined by the data structure that implements it. This choice is what makes comparison possible: an embedding index, a temporal knowledge graph [40, 2] and a table of rows can all satisfy the same postcondition, and a system that omits an operation cannot hide behind end-to-end accuracy. The full definitions, including variants and failure modes, are in `taxonomy.md`; Table 1 is the summary.

Table 2 maps seven existing systems onto the taxonomy. The result is the argument in one view: three operation families are covered by two of the seven systems, and the two cross-cutting axes are partial or absent in five of seven. Neither fact is visible in an end-to-end accuracy number.

Differences from the closest prior work. [27] orders the field by memory type and reviews metrics, but defines no runnable procedure. [60] decomposes the system into four internal modules and measures twelve systems, but its boundaries are internal: a third party cannot run that framework against a server it did not implement. [57] predates the memory-server ecosystem and describes operations per system rather than as testable postconditions. [33] evaluates MCP servers, but for tool-use effectiveness rather than memory semantics. [50] verifies one property (retrieval admissibility); we adopt it as a task family instead of duplicating it. This positioning is a claim of design difference, not of empirical superiority (claims C1–C5 in `claims.md`).

Operation	Postcondition (what must be true afterwards)	Evidence the interface must return
O1 WRITE	exactly one record created or mutated, carrying source, author, timestamp, derivation and lifecycle state; idempotent under a supplied key	record id plus a read-back of the stored record
O2 READ	returned set is relevant, *admissible* for the caller, and traceable	ordered (record id, score, provenance) list; dropped candidates reported separately from "no candidate existed"
O3 CONSOLIDATE	input batch replaced by fewer generalisations, each listing the ids it derives from	compression ratio, derivation links, probe set re-run after consolidation
O4 REVISE	conflict resolved explicitly (supersede / reject / unresolved), revision chain readable, unrelated records untouched	revision chain, hash of unchanged records, post-revision read
O5 INTEND	deferred intention survives context loss, fires exactly once at its cue with its context available	intention id, trigger spec, fire/no-fire record, context at fire time
O6 ASSERT	claim carries status and a confidence score with a declared elicitation method; the store can report its own gaps	status, score plus prompt, outcome, store-state self-report

Table 1: The six operations, their postconditions and the evidence that makes them checkable.

System	O1	O2	O3	O4	O5	O6	Prov.	Lifecycle
MemGPT / Letta [37, 12]	+	+	∼	∼	−	∼	∼	−
Generative Agents [38]	+	+	+	−	−	−	∼	−
MemoryBank [58]	+	+	+	−	−	−	−	∼
A-MEM [54]	+	+	∼	∼	−	−	∼	−
Zep / Graphiti [40, 2]	+	+	∼	+	−	−	+	+
Mem0 [20, 1]	+	+	+	∼	−	−	∼	∼
Liquid / Epistemic / Dream / Prospective [5, 4, 3, 7]	+	+	+	+	+	+	∼	+

Table 2: Operation coverage of seven systems. + present and observable, ∼ partially present, − absent.

4 Evaluation protocol

The protocol specifies task families, metrics, a run procedure and a report format; the schema and validator ship with the paper, so a report that omits a mandatory field is rejected rather than published.

Task families. T1 write fidelity; T2 read accuracy and ranking; T3 temporal reasoning; T4 admissibility (principal, policy, lifecycle); T5 update and conflict; T6 consolidation retention (identical probe set before and after); T7 prospective memory (cue firing, suspend/resume, cross-session persistence); T8 epistemic status and calibration; T9 durability and determinism across restarts; T10 adversarial and privacy. Rationale: T4 follows the finding that recall metrics cannot detect inadmissible retrieval [50]; T6 exists separately from T2 because retention failure precedes retrieval [42]; T7 is adopted from prospective-memory benchmarking [31, 21]; T8 requires status, elicitation method and outcome to be stored together [36, 56]; T10 follows demonstrated memory poisoning and membership inference [45, 18].

Metrics and who computes them. Each metric is defined by a formula, a unit and a computer: the runner harness, an independent programmatic check over returned records and store state, or — only where a program cannot judge — a two-grader rubric with Cohen's κ. No language-model judge is used for a metric that code can compute. Headline metrics include write precision/recall and duplicate rate; recall@k and MRR; provenance correctness; admissibility error rate; context inflation (retrieved characters per required character, the protocol's proxy in a run with no tokenizer); probe survival and compression ratio; contradiction detection F_1, revision correctness and **stale-read rate**; cue hit rate and false-fire rate; expected calibration error [25] *together with* discrimination (AUROC), since neither alone is informative [51]; and cost in tokens and USD against a price table whose date is part of the report.

Procedure. Servers are selected by a stated rule and every candidate, included or not, is listed, to make the sample auditable [16]. Each run pins the server commit, the MCP revision, the transport, the runtime, the model parameters and the seed set; runs are repeated at least three times, and dispersion is reported. Evidence position [32, 26] and the context budget [52] are controlled and recorded. Adversarial streams run last. Failure codes (server error, timeout, budget exhausted, not implemented, nondeterministic, leakage, stale read, lost intention) are recorded outcomes, not missing rows.

Report and disclosure. The report is JSON, validated against `protocol_schema.json`; it carries protocol version, subject identity including commit SHA, runner configuration, budgets with a dated price table, per-family metrics with bootstrap confidence intervals, raw log paths so every number can be recomputed, failure counts, and a mandatory disclosure block (selection rule, candidates, prompts hash, environment, known deviations) motivated by the finding that benchmark papers systematically omit exactly those fields [35]. The format is exercised end to end by the two reports of Section 6: each passes the validator, and each carries the candidate list with the ground for every inclusion and exclusion, plus its own list of stated deviations.

5 Reference implementation

The reference implementation ([8]) exposes five of the six operations as public MCP servers: liquid memory with contradiction handling and revision chains [5], epistemic status with calibrated confidence [4], offline consolidation as "dreaming" [3], prospective memory with intention stacks and session snapshots [7], and a role-based review swarm containing provenance, temporal and uncertainty roles [6]. Commit-pinned facts (tool counts, file inventory, last commit) were collected read-only into `data/repo-facts.json`.

The same artefact carries the paper's known gaps. The operations live in separate processes, so a record's provenance is not shared automatically. Derivation is carried as free text rather than an exportable PROV-O graph [9]. Cost instrumentation (tokens, latency) is not implemented, although the literature treats cost as a first-class design variable [43, 55]. Baselines exist only where a server could be launched and measured: of the servers above, the liquid store is measured in Section 6, while the epistemic, dreaming and prospective servers could not be launched on the measurement host and remain unmeasured. These gaps are precisely why the protocol makes cost, provenance and restart probes mandatory rather than optional.

6 Worked comparison: two memory servers under one protocol

The pair of runs. Two servers from the reference set were exercised with the protocol of Section 4: sint-liquid-mcp and sint-memory-mcp, each pinned to a commit (Table 3), each over 10 task families, 3 repetitions per family and the seed set 1, 2, 3 (81 tasks per repetition), driven over MCP stdio by the harness named in the same table. Both reports pass the schema validator,

and every number below is read from them. This section is a *pair of profiles*, not a ranking: the subjects differ in storage design by construction — one is a belief store with revisions, the other an append-only hash-chained block store, as the candidate list in each report states — and the sample is two servers.

How the tables were produced. Every table in this section is generated from the two `report.json` files by `refslint tables`, and every number in the sentences is a macro defined in the same generated file; `data/run-numbers.tsv` lists each value with the JSON path it came from. A number that has no field in a report cannot be printed here, and `refslint tables -check` fails if the generated files no longer match the reports. Values are shown rounded to at most four decimals. *null* is a metric that the protocol defines and that the subject's tool surface cannot produce, with the reason in the report's own note for that metric; n is the number of repetitions the metric could be computed over. Table 3 gives the identity of the two runs, Tables 4–13 the per-family profiles, and Table 14 the same reading pooled by taxonomy operation.

What is invariant across the two subjects. The protocol reproduces identical readings where the mechanism is the same. Durability and determinism are complete in both: restart survival 1.0, snapshot-reopen survival 1.0 and repeated-query equality 1.0. Provenance correctness of returned records is 1.0 in both, abstention on out-of-coverage questions 1.0 in both, and in both the store's self-reported counters agree with the harness write log (monitoring accuracy 1.0). Query-only interactions changed no later answer for the target record (poisoning influence 0.0 in both). Membership inference against a record's own writer succeeds in both (1.0; the same principal wrote the records, so this is store-level inferability, not cross-principal leakage). Two families are *not implemented in either subject*: no advertised tool accepts an as-of date (`temporal_correctness` null) or a caller identity (`principal_scoping_exposed` 0.0), recorded as 15 and 24 `not_implemented` failures respectively. Both expose timestamps on returned records (`temporal_metadata_exposed` 1.0) but no lifecycle field (`lifecycle_state_exposed` 0.0), and the policy check over returned records — the tool surface has no caller parameter — gives the same admissibility error rate in both, 0.5.

Where the profiles diverge, and in which direction. On the write path the two subjects are not interchangeable. Write precision is 1.0 for sint-liquid-mcp and 0.75 for sint-memory-mcp, and the shortfall is fully accounted for by retries: the duplicate rate is 0.0 against 0.25, and the programmatic check counts 4.0 idempotency violations against 0.0 — at equal write recall 1.0 in both, so nothing was lost and four records were stored twice under repeated intent. Read-side ranking runs the other way: recall@5 is 1.0 for sint-memory-mcp against 0.5, and MRR 0.75 ([0.6806, 0.8056]) against 0.5. The pair is informative precisely because one run inverts the other: the subject with the cleaner write path is the one that misses a topic-only query (`topic_query_hit_rate` 0.0 against 1.0), and the subject that returns 2.5 times the required characters is the one that leaves duplicate records behind. Write latency is close (17.8175 ms against 16.4423 ms at p50); read latency is not (13.7778 ms against 102.8932 ms at p95), and is the weakest number here: wall clock around a stdio call on one workstation under unspecified load, with three repetitions.

Revision: an operation-level difference that retrieval scores do not show. In T5 the harness writes contradictory pairs and then asks what survives. Neither subject flags a conflict by itself: `contradiction_detection_f1` is 0.0 in both, and since the harness never names the conflicting pair, this measures detection rather than registration. sint-liquid-mcp then applies the revision it is asked for (revision correctness 1.0, revision-chain completeness 1.0) and leaves unrelated records untouched (collateral change 0.0) — yet its read path still serves the superseded

content (stale-read rate 1.0) and returns both competing contents in 1.0 of the conflicts. sint-memory-mcp is append-only, so it has no revision to be correct about (revision correctness *null*, chain completeness 0.0), and it too serves the superseded content (1.0), with both contents returned in 1.0 of the conflicts. Read together, the two profiles say one thing about revision as an interface: a store can accept a revision and still serve the superseded record, and T5 sees that difference where a retrieval score would not. We state it as an observation of two runs, not as a property of either design.

Prospective memory and calibration: where the pair disagrees most. T7 asks the store for a fact through a cue-only query after a process restart; neither subject has an intention surface, so firing is operationalised as store-mediated cue retrieval. The cue hit rate is 0.0 for sint-liquid-mcp and 1.0 for sint-memory-mcp, with false fires 0.0 in both; the stored context is returned with the fire in 1.0 of the fires of sint-memory-mcp, and the same metric is *null* for sint-liquid-mcp only because there is no fire to check. In T8 both halves of calibration are reported and they do not agree: expected calibration error 0.3667 against 0.4333, discrimination (AUROC) 0.75 against 0.3, with top-ranked answer correctness 0.5 against 0.75. The confidence reported here is the highest confidence among returned records (Section 4), so these are readings of the stores' own confidence fields, not of verbalised model confidence. With 3 repetitions and one task set per family, a pair this far apart is an observation to be re-measured on more subjects before anything is said about calibration in general.

Hypotheses stated before the run. H1 — explicit lifecycle metadata buys a lower stale-read rate at a measurable write-latency cost — is not testable on this pair: neither subject exposes a lifecycle field on returned records, so the antecedent is absent, and both serve stale content at the same rate 1.0. H2 — a subject without a prospective-memory operation fails T7 entirely while its T2 and T6 scores remain high — holds for one of the two: sint-liquid-mcp fails the cue-only family (0.0) while scoring read recall 0.5 and *null* on the consolidation family it does not implement; sint-memory-mcp answers the cue query (1.0) through its substring read path. One subject consistent with a hypothesis and one not is what a two-subject sample can produce; the hypothesis is neither confirmed nor refuted, and its status in `claims.md` is unchanged.

What this comparison does not measure. Token and cost metrics are *null* by construction in both reports: no model is in the loop, so there is no token accounting and no price table applies, as the reports state in `budgets.price_table`. Neither subject was measured against a third-party baseline, and neither was measured for availability, concurrency or behaviour under load. Consolidation (O3) is *null* for sint-liquid-mcp, which has no maintenance operation, and only partly populated for sint-memory-mcp, where an offline decay pass stood in for the consolidation the protocol asks for (probe survival 1.0, compression ratio 1.0); the consolidation-specific metrics stay *null* in both. The one rubric metric (`abstraction_faithfulness`) was never graded, so no κ is reported. A *null* therefore means one of two different things — the operation does not exist, or it exists but the metric has nothing to score — and each report's own note says which.

Table 3: Identity of the two runs: subject, pinned revision, runtime, budgets and the raw responses every number of this section is computed from. Every value is read from the corresponding `report.json`; the table is generated by `refslint tables`.

field	sint-liquid-mcp	sint-memory-mcp
subject	sint-liquid-mcp	sint-memory-mcp
repository	`https://github.com/kostyk348/sint-liquid-mcp`	`https://github.com/kostyk348/sint`

field	sint-liquid-mcp	sint-memory-mcp
commit	bfffce2cdec35c11aa0d90ae02689d69df596b11	7554f7eef853cebafae34e6654acc9871fb01163
published version	MCP server as published in the repository snapshot (no release tag)	MCP server as published in the repository snapshot (no release tag)
transport / MCP revision	stdio / 2025-03-26	stdio / 2025-03-26
runtime	Python 3.14.7, mcp 1.9.4 (mcp.server.fastmcp)	Python 3.14.7, mcp 1.9.4 (mcp.server.fastmcp)
deployment	local process, fresh data directory per repetition (USERPROFILE/HOME redirected)	local process, fresh data directory per repetition (USERPROFILE/HOME redirected)
store kept in the run dir	reports/sint-liquid-mcp-2026-10-16/data/seed<seed>	reports/sint-memory-mcp-2026-10-16/data/seed<seed>
model under test	none (store-only)	none (store-only)
hardware	AMD64, 4 logical CPUs (workstation class, not a benchmark host)	AMD64, 4 logical CPUs (workstation class, not a benchmark host)
runner / harness	uncommitted working tree: sha256: 9b79d0aa12ae1660 over code/run_protocol.py + code/streams.py + code/servers.json (this workspace has no git metadata, so there is no commit to name)	uncommitted working tree: sha256: 9b79d0aa12ae1660 over code/run_protocol.py + code/streams.py + code/servers.json (this workspace has no git metadata, so there is no commit to name)
environment	Windows-11-10.0.26200-SP0; Intel64 Family 6 Model 140 Stepping 2, GenuineIntel; server interpreter C:/Users/Lab/AppData/Local/hermes/installs/2636d87a1639ae64/environments/f32597c25d734363b2d220b038061d05/venv/Scripts/python.exe; harness Python 3.14.7 (stdlib only)	Windows-11-10.0.26200-SP0; Intel64 Family 6 Model 140 Stepping 2, GenuineIntel; server interpreter C:/Users/Lab/AppData/Local/hermes/installs/2636d87a1639ae64/environments/f32597c25d734363b2d220b038061d05/venv/Scripts/python.exe; harness Python 3.14.7 (stdlib only)
repetitions / seeds	3 / 1, 2, 3	3 / 1, 2, 3
budgets per repetition	400 calls, 900 s wall clock	400 calls, 900 s wall clock
confidence intervals	bootstrap_percentile, 1000 resamples	bootstrap_percentile, 1000 resamples
failures recorded	not_implemented x 15	not_implemented x 24
raw responses	reports/sint-liquid-mcp-2026-10-16/raw (33 JSONL files)	reports/sint-memory-mcp-2026-10-16/raw (33 JSONL files)
report.json	reports/sint-liquid-mcp-2026-10-16/report.json	reports/sint-memory-mcp-2026-10-16/report.json

Table 4: Task family T1: every metric the protocol defines for it that `report.json` carries, for both subjects, read side by side (12 tasks per repetition, 3 repetitions, seeds 1, 2, 3). Each subject cell gives the value, its 95% bootstrap CI and n; *null* is a metric that the protocol defines and that the subject's tool surface cannot produce, with the reason in the report's own note for that metric. Values are printed rounded to at most four decimals; the unrounded values and their JSON paths are in `data/run-numbers.tsv`. Generated by `refslint tables`.

metric	sint-liquid-mcp	sint-memory-mcp	computer
`write_precision` (*ratio*)	1.0 [1.0, 1.0], n=3	0.75 [0.75, 0.75], n=3	harness

metric	sint-liquid-mcp	sint-memory-mcp	computer
write_recall (*ratio*)	1.0 [1.0, 1.0], n=3	1.0 [1.0, 1.0], n=3	harness
duplicate_rate (*ratio*)	0.0 [0.0, 0.0], n=3	0.25 [0.25, 0.25], n=3	harness
idempotency_ violations (*count*)	0.0 [0.0, 0.0], n=3	4.0 [4.0, 4.0], n=3	programmatic_ check
write_latency_ p50_ms (*ms*)	17.8175 [17.0155, 18.4985], n=3	16.4423 [15.167, 17.2955], n=3	harness
write_latency_ p95_ms (*ms*)	21.0234 [19.3115, 21.8968], n=3	18.9019 [17.329, 20.2025], n=3	harness
tokens_per_ write (*tokens*)	*null* (n=0)	*null* (n=0)	harness
cost_per_1000_ writes_usd (*usd*)	*null* (n=0)	*null* (n=0)	harness

Table 5: Task family T2: every metric the protocol defines for it that `report.json` carries, for both subjects, read side by side (6 tasks per repetition, 3 repetitions, seeds 1, 2, 3). Each subject cell gives the value, its 95% bootstrap CI and n; *null* is a metric that the protocol defines and that the subject's tool surface cannot produce, with the reason in the report's own note for that metric. Values are printed rounded to at most four decimals; the unrounded values and their JSON paths are in `data/run-numbers.tsv`. Generated by `refslint tables`.

metric	sint-liquid-mcp	sint-memory-mcp	computer
recall_at_5 (*ratio*)	0.5 [0.5, 0.5], n=3	1.0 [1.0, 1.0], n=3	harness
required_ record_hit_rate (*ratio*)	0.5 [0.5, 0.5], n=3	1.0 [1.0, 1.0], n=3	harness
mrr (*ratio*)	0.5 [0.5, 0.5], n=3	0.75 [0.6806, 0.8056], n=3	harness
key_query_hit_ rate (*ratio*)	1.0 [1.0, 1.0], n=3	1.0 [1.0, 1.0], n=3	harness
topic_query_ hit_rate (*ratio*)	0.0 [0.0, 0.0], n=3	1.0 [1.0, 1.0], n=3	harness
provenance_ correctness (*ratio*)	1.0 [1.0, 1.0], n=3	1.0 [1.0, 1.0], n=3	harness
context_ inflation (*ratio*)	0.5 [0.5, 0.5], n=3	2.5 [2.5, 2.5], n=3	harness
read_latency_ p50_ms (*ms*)	4.4998 [4.1825, 4.9225], n=3	8.0937 [7.6565, 8.5435], n=3	harness
read_latency_ p95_ms (*ms*)	13.7778 [12.8863, 14.3423], n=3	102.8932 [101.5778, 104.9318], n=3	harness

Table 6: Task family T3: every metric the protocol defines for it that `report.json` carries, for both subjects, read side by side (4 tasks per repetition, 3 repetitions, seeds 1, 2, 3). Each subject cell gives the value, its 95% bootstrap CI and n; *null* is a metric that the protocol defines and that the subject's tool surface cannot produce, with the reason in the report's own note for that metric. Values are printed rounded to at most four decimals; the unrounded values and their JSON paths are in `data/run-numbers.tsv`. Generated by `refslint tables`.

metric	sint-liquid-mcp	sint-memory-mcp	computer
`temporal_correctness` (*ratio*)	*null* (n=0)	*null* (n=0)	harness
`temporal_metadata_exposed` (*ratio*)	1.0 [1.0, 1.0], n=3	1.0 [1.0, 1.0], n=3	programmatic_check

Table 7: Task family T4: every metric the protocol defines for it that `report.json` carries, for both subjects, read side by side (6 tasks per repetition, 3 repetitions, seeds 1, 2, 3). Each subject cell gives the value, its 95% bootstrap CI and n; *null* is a metric that the protocol defines and that the subject's tool surface cannot produce, with the reason in the report's own note for that metric. Values are printed rounded to at most four decimals; the unrounded values and their JSON paths are in `data/run-numbers.tsv`. Generated by `refslint tables`.

metric	sint-liquid-mcp	sint-memory-mcp	computer
`admissibility_error_rate` (*ratio*)	0.5 [0.5, 0.5], n=3	0.5 [0.5, 0.5], n=3	programmatic_check
`lifecycle_state_exposed` (*ratio*)	0.0 [0.0, 0.0], n=3	0.0 [0.0, 0.0], n=3	programmatic_check
`principal_scoping_exposed` (*ratio*)	0.0 [0.0, 0.0], n=3	0.0 [0.0, 0.0], n=3	programmatic_check

Table 8: Task family T5: every metric the protocol defines for it that `report.json` carries, for both subjects, read side by side (4 tasks per repetition, 3 repetitions, seeds 1, 2, 3). Each subject cell gives the value, its 95% bootstrap CI and n; *null* is a metric that the protocol defines and that the subject's tool surface cannot produce, with the reason in the report's own note for that metric. Values are printed rounded to at most four decimals; the unrounded values and their JSON paths are in `data/run-numbers.tsv`. Generated by `refslint tables`.

metric	sint-liquid-mcp	sint-memory-mcp	computer
`contradiction_detection_f1` (*ratio*)	0.0 [0.0, 0.0], n=3	0.0 [0.0, 0.0], n=3	harness

metric	sint-liquid-mcp	sint-memory-mcp	computer
`revision_correctness` (*ratio*)	1.0 [1.0, 1.0], n=3	*null* (n=0)	programmatic_check
`chain_completeness` (*ratio*)	1.0 [1.0, 1.0], n=3	0.0 [0.0, 0.0], n=3	programmatic_check
`stale_read_rate` (*ratio*)	1.0 [1.0, 1.0], n=3	1.0 [1.0, 1.0], n=3	harness
`both_competing_contents_served_rate` (*ratio*)	1.0 [1.0, 1.0], n=3	1.0 [1.0, 1.0], n=3	harness
`policy_winner_top_ranked_rate` (*ratio*)	*null* (n=0)	0.5 [0.5, 0.5], n=3	harness
`collateral_change_rate` (*ratio*)	0.0 [0.0, 0.0], n=3	0.0 [0.0, 0.0], n=3	programmatic_check

Table 9: Task family T6: every metric the protocol defines for it that `report.json` carries, for both subjects, read side by side (4 tasks per repetition, 3 repetitions, seeds 1, 2, 3). Each subject cell gives the value, its 95% bootstrap CI and n; *null* is a metric that the protocol defines and that the subject's tool surface cannot produce, with the reason in the report's own note for that metric. Values are printed rounded to at most four decimals; the unrounded values and their JSON paths are in `data/run-numbers.tsv`. Generated by `refslint tables`.

metric	sint-liquid-mcp	sint-memory-mcp	computer
`probe_survival_rate` (*ratio*)	*null* (n=0)	1.0 [1.0, 1.0], n=3	harness
`compression_ratio` (*ratio*)	*null* (n=0)	1.0 [1.0, 1.0], n=3	harness
`derived_from_completeness` (*ratio*)	*null* (n=0)	*null* (n=0)	programmatic_check
`abstraction_faithfulness` (*score_1_5*)	*null* (n=0)	*null* (n=0)	rubric_two_graders
`cost_per_consolidation_usd` (*usd*)	*null* (n=0)	*null* (n=0)	harness

Table 10: Task family T7: every metric the protocol defines for it that `report.json` carries, for both subjects, read side by side (3 tasks per repetition, 3 repetitions, seeds 1, 2, 3). Each subject cell gives the value, its 95% bootstrap CI and n; *null* is a metric that the protocol defines and that the subject's tool surface cannot produce, with the reason in the report's own note for that metric. Values are printed rounded to at most four decimals; the unrounded values and their JSON paths are in `data/run-numbers.tsv`. Generated by `refslint tables`.

metric	sint-liquid-mcp	sint-memory-mcp	computer
`cue_hit_rate` (*ratio*)	0.0 [0.0, 0.0], n=3	1.0 [1.0, 1.0], n=3	harness
`false_fire_rate` (*ratio*)	0.0 [0.0, 0.0], n=3	0.0 [0.0, 0.0], n=3	harness
`context_availability_at_fire` (*ratio*)	*null* (n=0)	1.0 [1.0, 1.0], n=3	programmatic_check
`fire_latency_calls` (*count*)	*null* (n=0)	0.0 [0.0, 0.0], n=3	harness

Table 11: Task family T8: every metric the protocol defines for it that `report.json` carries, for both subjects, read side by side (12 tasks per repetition, 3 repetitions, seeds 1, 2, 3). Each subject cell gives the value, its 95% bootstrap CI and n; *null* is a metric that the protocol defines and that the subject's tool surface cannot produce, with the reason in the report's own note for that metric. Values are printed rounded to at most four decimals; the unrounded values and their JSON paths are in `data/run-numbers.tsv`. Generated by `refslint tables`.

metric	sint-liquid-mcp	sint-memory-mcp	computer
`abstention_correctness` (*ratio*)	1.0 [1.0, 1.0], n=3	1.0 [1.0, 1.0], n=3	harness
`in_coverage_answer_correctness` (*ratio*)	0.5 [0.5, 0.5], n=3	0.75 [0.75, 0.75], n=3	harness
`key_question_correctness` (*ratio*)	1.0 [1.0, 1.0], n=3	1.0 [1.0, 1.0], n=3	harness
`subject_question_correctness` (*ratio*)	0.0 [0.0, 0.0], n=3	0.5 [0.5, 0.5], n=3	harness
`ece` (*ratio*)	0.3667 [0.3667, 0.3667], n=3	0.4333 [0.4333, 0.4333], n=3	harness
`discrimination_auroc` (*ratio*)	0.75 [0.75, 0.75], n=3	0.3 [0.3, 0.3], n=3	harness
`monitoring_accuracy` (*ratio*)	1.0 [1.0, 1.0], n=3	1.0 [1.0, 1.0], n=3	programmatic_check

Table 12: Task family T9: every metric the protocol defines for it that `report.json` carries, for both subjects, read side by side (18 tasks per repetition, 3 repetitions, seeds 1, 2, 3). Each subject cell gives the value, its 95% bootstrap CI and n; *null* is a metric that the protocol defines and that the subject's tool surface cannot produce, with the reason in the report's own note for that metric. Values are printed rounded to at most four decimals; the unrounded values and their JSON paths are in `data/run-numbers.tsv`. Generated by `refslint tables`.

metric	sint-liquid-mcp	sint-memory-mcp	computer
`restart_survival` (*ratio*)	1.0 [1.0, 1.0], n=3	1.0 [1.0, 1.0], n=3	harness
`snapshot_reopen_survival` (*ratio*)	1.0 [1.0, 1.0], n=3	1.0 [1.0, 1.0], n=3	harness
`repeat_query_equality` (*ratio*)	1.0 [1.0, 1.0], n=3	1.0 [1.0, 1.0], n=3	harness

Table 13: Task family T10: every metric the protocol defines for it that `report.json` carries, for both subjects, read side by side (12 tasks per repetition, 3 repetitions, seeds 1, 2, 3). Each subject cell gives the value, its 95% bootstrap CI and n; *null* is a metric that the protocol defines and that the subject's tool surface cannot produce, with the reason in the report's own note for that metric. Values are printed rounded to at most four decimals; the unrounded values and their JSON paths are in `data/run-numbers.tsv`. Generated by `refslint tables`.

metric	sint-liquid-mcp	sint-memory-mcp	computer
`poisoning_influence_rate` (*ratio*)	0.0 [0.0, 0.0], n=3	0.0 [0.0, 0.0], n=3	harness
`membership_tpr` (*ratio*)	1.0 [1.0, 1.0], n=3	1.0 [1.0, 1.0], n=3	harness
`membership_fpr` (*ratio*)	0.0 [0.0, 0.0], n=3	0.0 [0.0, 0.0], n=3	harness
`membership_inference_advantage` (*ratio*)	1.0 [1.0, 1.0], n=3	1.0 [1.0, 1.0], n=3	harness

Table 14: The same profile by taxonomy operation: each operation's metrics pooled over the repetitions of the families that exercise it (the report states which), for both subjects side by side. `supported` is the subject's own verdict for that operation. Generated by `refslint tables`.

metric	sint-liquid-mcp	sint-memory-mcp	computer
`write_precision` (*ratio*)	1.0 [1.0, 1.0], n=3	0.75 [0.75, 0.75], n=3	harness
`write_recall` (*ratio*)	1.0 [1.0, 1.0], n=3	1.0 [1.0, 1.0], n=3	harness
`duplicate_rate` (*ratio*)	0.0 [0.0, 0.0], n=3	0.25 [0.25, 0.25], n=3	harness

metric	sint-liquid-mcp	sint-memory-mcp	computer
write_latency_p50_ms (*ms*)	17.8175 [17.3232, 18.4985], n=3	16.4423 [15.167, 17.2955], n=3	harness
write_latency_p95_ms (*ms*)	21.0234 [19.3115, 21.8968], n=3	18.9019 [17.329, 20.2025], n=3	harness
tokens_per_write (*tokens*)	*null* (n=0)	*null* (n=0)	harness
cost_per_1000_writes_usd (*usd*)	*null* (n=0)	*null* (n=0)	harness
idempotency_violations (*count*)	0.0 [0.0, 0.0], n=3	4.0 [4.0, 4.0], n=3	programmatic_check
recall_at_5 (*ratio*)	0.5 [0.5, 0.5], n=3	1.0 [1.0, 1.0], n=3	harness
mrr (*ratio*)	0.5 [0.5, 0.5], n=3	0.75 [0.6806, 0.8056], n=3	harness
required_record_hit_rate (*ratio*)	0.5 [0.5, 0.5], n=3	1.0 [1.0, 1.0], n=3	harness
provenance_correctness (*ratio*)	1.0 [1.0, 1.0], n=3	1.0 [1.0, 1.0], n=3	harness
admissibility_error_rate (*ratio*)	0.5 [0.5, 0.5], n=3	0.5 [0.5, 0.5], n=3	programmatic_check
context_inflation (*ratio*)	0.5 [0.5, 0.5], n=3	2.5 [2.5, 2.5], n=3	harness
read_latency_p50_ms (*ms*)	4.4998 [4.1825, 4.9225], n=3	8.0937 [7.6565, 8.5435], n=3	harness
read_latency_p95_ms (*ms*)	13.7778 [12.8863, 14.3423], n=3	102.8932 [101.5778, 104.9318], n=3	harness
compression_ratio (*ratio*)	*null* (n=0)	1.0 [1.0, 1.0], n=3	harness
probe_survival_rate (*ratio*)	*null* (n=0)	1.0 [1.0, 1.0], n=3	harness
derived_from_completeness (*ratio*)	*null* (n=0)	*null* (n=0)	programmatic_check
cost_per_consolidation_usd (*usd*)	*null* (n=0)	*null* (n=0)	harness
contradiction_detection_f1 (*ratio*)	0.0 [0.0, 0.0], n=3	0.0 [0.0, 0.0], n=3	harness
revision_correctness (*ratio*)	1.0 [1.0, 1.0], n=3	*null* (n=0)	programmatic_check
chain_completeness (*ratio*)	1.0 [1.0, 1.0], n=3	0.0 [0.0, 0.0], n=3	programmatic_check
stale_read_rate (*ratio*)	1.0 [1.0, 1.0], n=3	1.0 [1.0, 1.0], n=3	harness
collateral_change_rate (*ratio*)	0.0 [0.0, 0.0], n=3	0.0 [0.0, 0.0], n=3	programmatic_check
cue_hit_rate (*ratio*)	0.0 [0.0, 0.0], n=3	1.0 [1.0, 1.0], n=3	harness

metric	sint-liquid-mcp	sint-memory-mcp	computer
false_fire_rate (*ratio*)	0.0 [0.0, 0.0], n=3	0.0 [0.0, 0.0], n=3	harness
fire_latency_calls (*count*)	*null* (n=0)	0.0 [0.0, 0.0], n=3	harness
context_availability_at_fire (*ratio*)	*null* (n=0)	1.0 [1.0, 1.0], n=3	programmatic_check
abstention_correctness (*ratio*)	1.0 [1.0, 1.0], n=3	1.0 [1.0, 1.0], n=3	harness
in_coverage_answer_correctness (*ratio*)	0.5 [0.5, 0.5], n=3	0.75 [0.75, 0.75], n=3	harness
key_question_correctness (*ratio*)	1.0 [1.0, 1.0], n=3	1.0 [1.0, 1.0], n=3	harness
subject_question_correctness (*ratio*)	0.0 [0.0, 0.0], n=3	0.5 [0.5, 0.5], n=3	harness
ece (*ratio*)	0.3667 [0.3667, 0.3667], n=3	0.4333 [0.4333, 0.4333], n=3	harness
discrimination_auroc (*ratio*)	0.75 [0.75, 0.75], n=3	0.3 [0.3, 0.3], n=3	harness
monitoring_accuracy (*ratio*)	1.0 [1.0, 1.0], n=3	1.0 [1.0, 1.0], n=3	programmatic_check
provenance_correctness (*ratio*)	1.0 [1.0, 1.0], n=3	1.0 [1.0, 1.0], n=3	harness
derived_from_completeness (*ratio*)	*null* (n=0)	*null* (n=0)	programmatic_check
lifecycle_state_exposed (*ratio*)	0.0 [0.0, 0.0], n=3	0.0 [0.0, 0.0], n=3	programmatic_check
admissibility_error_rate (*ratio*)	0.5 [0.5, 0.5], n=3	0.5 [0.5, 0.5], n=3	programmatic_check
stale_read_rate (*ratio*)	1.0 [1.0, 1.0], n=3	1.0 [1.0, 1.0], n=3	harness
restart_survival (*ratio*)	1.0 [1.0, 1.0], n=3	1.0 [1.0, 1.0], n=3	harness
snapshot_reopen_survival (*ratio*)	1.0 [1.0, 1.0], n=3	1.0 [1.0, 1.0], n=3	harness

7 Limitations

Full list with statuses in `claims.md`. The following are the limits of the measurement reported in Section 6, placed with the results rather than after them.

- **Two subjects, one provenance.** Both measured servers come from one author's reference set. The selection rule and every candidate — included or excluded, with the ground for each decision — are in both reports, including the host constraint (package installation failed with

`Errno 28`, disk full, so only one MCP framework version was importable). The sample selects for servers that launch on this host and is not representative of the MCP ecosystem.

- **Three repetitions.** 3 repetitions per family at seeds 1, 2, 3, the protocol minimum, is the only axis of variation: the reported intervals are percentile bootstrap intervals over that axis (bootstrap_percentile) and are coarse by construction. They measure run-to-run dispersion, not task-sampling variability, so a difference smaller than the interval width should not be read as a difference between systems.
- **One machine.** Both runs come from the single environment recorded in Table 3: a workstation, not a benchmark host, with AMD64, 4 logical CPUs (workstation class, not a benchmark host). Latencies are wall clock around a stdio call and depend on machine load; no latency claim is made beyond this host.
- **No model in the loop.** The runs are store-only: token and USD metrics are *null* by construction in both reports, so the protocol's cost axis is defined and validated but untested. Any statement that costs change which memory design looks preferable remains a position (claim C5).
- **The harness is Python.** The reports were produced by `code/run_protocol.py` (uncommitted working tree: sha256:9b79d0aa12ae1660 over code/run_protocol.py + code/streams.py + code/servers.json (this workspace has no git metadata, so there is no commit to name)). The package's tooling rule — new utilities in Rust, one binary with subcommands — is met by the validation, bibliography and table-generation pipeline (`tools/refslint`), but the runner itself is still the card-named Python harness, because it has to interoperate with the MCP Python SDK that both subjects import. It is identified by a content hash rather than a commit, because this workspace carries no git metadata. There is no offline command that rebuilds a `report.json` from its raw logs: the metric layer lives inside that runner, and re-running the protocol requires the host setup described in the package README. What is reproducible with one command is everything downstream of the reports: validation, the tables, and every number in this paper.
- **Coverage of the families.** T3 (as-of-date reads) and T4 (principal-scoped reads) are not implemented in either subject, so those families contribute exposure metrics (timestamp exposure, lifecycle-field exposure, principal scoping across advertised tool schemas) rather than the operations the taxonomy defines. T6 is mostly *null*. The profiles are therefore denser for O1, O2, O5, O6 and the LIFECYCLE axis than for O3 and O4.
- **Rubric metrics.** No double grading was performed and no κ value exists, so the rubric-scored metric remains exploratory (claim D5).
- **Taxonomy and transfer.** The taxonomy's completeness is not claimed; several store-internal steps (extraction, consolidation) may be performed by models we cannot pin and are reported as unspecified rather than guessed; and the domain-transfer claim to industrial settings is untested, with [46] cited as the closest precedent for executable domain evaluation rather than as evidence of transfer (claims D1–D7).

8 Conclusion

The reusable parts of this work are an operation checklist, a protocol whose reports are machine-validated, a reference implementation with its gaps disclosed, a reference base in which every source was resolved by a script, and a generated set of tables that ties every number in the comparison to a field in a validated report. The first pair of reports now exists and is reported with its limits: the protocol separates two memory servers on the write path, reproduces the

same readings where the mechanism is the same, and surfaces a revision difference that retrieval scores hide. The next step is a third and a fourth subject outside one author's reference set, a model in the loop so that the token and cost metrics stop being *null*, and more repetitions than the protocol minimum.

Data and code availability

The two runs are archived with the paper as the two run directories named in Table 3, each beside its raw MCP stdio logs (33 JSONL files per run, every request, response and latency) and the per-repetition store directories the runs wrote, so that the reported numbers can be recomputed from the responses. The measured systems are https://github.com/kostyk348/sint and https://github.com/kostyk348/sint-liquid-mcp, at the commits pinned in the same table; the runner is identified there by its content hash. The two runs issued 681 and 660 MCP calls against advertised tool surfaces of 8 and 19 tools respectively (counts taken from each report's `statistics` block).

Four commands reproduce what is claimed here:

1. `bashbuild.sh` builds this PDF; it regenerates the bibliography from the curated source list, checks every citation key, regenerates the comparison tables and refuses to continue if a citation or a number lookup does not resolve.

2. Validate both reports against the published schema: `refslintreport--schemacode/protocol_schema.jsonreports/*/report.json`. `refslintselftest` re-derives five deliberately broken reports and asserts that all five are rejected.

3. `refslinttables--root.--check` rebuilds the result tables, the number macros and the number ledger from the two reports and fails if the files on disk differ from them, or if this paper uses a number for which no report field exists.

4. `bashverify_all.sh` runs the whole integrity set — reference base, citation keys, report validation, negative tests, JSON parse, port parity, table freshness, PDF rebuild — and prints `ALL CHECKS PASSED` or fails.

The pipeline utility is `tools/refslint` (Rust, standard library only, built by cargo; version 0.1.0 declared in `tools/refslint/Cargo.toml`), which also generates the bibliography, the reference-verification table and the report validation. The three documents that carry the unabridged material — `taxonomy.md`, `protocol.md` and `related-work.md` — ship in the same folder, as does `claims.md`, the ledger of every claim with the command or source that supports it.

Acknowledgements

Artificial-intelligence assistance (literature retrieval, metadata resolution, artefact collection and drafting support) was used in preparing this manuscript; every cited source was verified by machine queries and the verification table is part of the repository. The author takes responsibility for all claims. The author thanks the maintainers of the SINT repositories for the artefact base.

References

[1] Build ai apps that remember - mem0. `https://docs.mem0.ai/`. HTTP 200, retrieved via direct fetch on 2026-10-16T12:24:48.

[2] Github - getzep/graphiti: Build real-time knowledge graphs for ai agents ů github. https://github.com/getzep/graphiti. HTTP 200, retrieved via direct fetch on 2026-10-16T12:24:48.

[3] Github - kostyk348/sint-dream-mcp: Agent dreaming — offline consolidation between mcp sessions. ů github. https://github.com/kostyk348/sint-dream-mcp. HTTP 200, retrieved via direct fetch on 2026-10-16T12:24:48.

[4] Github - kostyk348/sint-epistemic-mcp: Epistemic status tracking mcp server — tracks what an agent knows vs in. https://github.com/kostyk348/sint-epistemic-mcp. HTTP 200, retrieved via direct fetch on 2026-10-16T12:24:48.

[5] Github - kostyk348/sint-liquid-mcp: Liquid memory — adaptive belief revision mcp server. ů github. https://github.com/kostyk348/sint-liquid-mcp. HTTP 200, retrieved via direct fetch on 2026-10-16T12:24:48.

[6] Github - kostyk348/sint-llm-swarm: Llm swarm with sint-intrinsic roles ů github. https://github.com/kostyk348/sint-llm-swarm. HTTP 200, retrieved via direct fetch on 2026-10-16T12:24:48.

[7] Github - kostyk348/sint-prospective-mcp: Prospective memory mcp server — tracks the agent's intentions, su. https://github.com/kostyk348/sint-prospective-mcp. HTTP 200, retrieved via direct fetch on 2026-10-16T12:24:48.

[8] Github - kostyk348/sint: Sint agent ecosystem — monorepo for all mcp servers, core memory system, and architec. https://github.com/kostyk348/sint. HTTP 200, retrieved via direct fetch on 2026-10-16T12:24:48.

[9] https://www.w3.org/tr/prov-o/. https://www.w3.org/TR/prov-o/. HTTP 403, retrieved via direct fetch on 2026-10-16T12:24:48.

[10] Introducing the model context protocol anthropic. https://www.anthropic.com/news/model-context-protocol. HTTP 200, retrieved via direct fetch on 2026-10-16T12:24:48.

[11] Json-rpc 2.0 specification. https://www.jsonrpc.org/specification. HTTP 200, retrieved via direct fetch on 2026-10-16T12:24:48.

[12] Letta documentation | letta docs. https://docs.letta.com/. HTTP 200, retrieved via direct fetch on 2026-10-16T12:24:48.

[13] Specification - model context protocol. https://modelcontextprotocol.io/specification/2025-06-18. HTTP 200, retrieved via direct fetch on 2026-10-16T12:24:48.

[14] Specification - model context protocol. https://modelcontextprotocol.io/specification/2025-03-26. HTTP 200, retrieved via direct fetch on 2026-10-16T12:24:48.

[15] What is the model context protocol (mcp)? - model context protocol. https://modelcontextprotocol.io/docs/getting-started/intro. HTTP 200, retrieved via direct fetch on 2026-10-16T12:24:48.

[16] Haseeb Mohammed Afsar. What a random draw from the mcp registry contains, and what tool-use benchmarks contain instead, 2026. arXiv:2609.10962 [cs.SE], first posted 2026-09-10; metadata verified via the arXiv API on 2026-10-16T12:24:48.

[17] John R. Anderson, Daniel Bothell, Michael D. Byrne, Scott Douglass, Christian Lebiere, and Yulin Qin. An integrated theory of the mind. *Psychological Review*, 111(4):1036–1060, 2004. DOI metadata verified via the Crossref REST API on 2026-10-16T12:24:48.

[18] Kai Chen, Yan Pang, and Tianhao Wang. Mrmmia: Membership inference attacks on memory in chat agents, 2026. arXiv:2605.27825 [cs.CR], first posted 2026-05-27; metadata verified via the arXiv API on 2026-10-16T12:24:48.

[19] Yihang Chen, Pin Qian, Su Wang, Sipeng Zhang, Huan Xu, Shuhuai Lin, and Xinpeng Wei. Does rag know when retrieval is wrong? diagnosing context compliance under knowledge conflict, 2026. arXiv:2605.14473 [cs.CL], first posted 2026-05-14; metadata verified via the arXiv API on 2026-10-16T12:24:48.

[20] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory, 2025. arXiv:2504.19413 [cs.CL], first posted 2025-04-28; metadata verified via the arXiv API on 2026-10-16T12:24:48.

[21] Gilles O. Einstein and Mark A. McDaniel. Normal aging and prospective memory. *Journal of Experimental Psychology: Learning, Memory, and Cognition*, 16(4):717–726, 1990. DOI metadata verified via the Crossref REST API on 2026-10-16T12:24:48.

[22] Faiq Miftakhul Falakh, Sebastian Rudolph, and Kai Sauerwald. A general katsuno-mendelzon-style characterization of agm belief base revision for arbitrary monotonic logics, 2021. arXiv:2104.14512 [cs.AI], first posted 2021-04-29; metadata verified via the arXiv API on 2026-10-16T12:24:48.

[23] Brian Falkenhainer. Towards a general-purpose belief maintenance system, 2013. arXiv:1304.3084 [cs.AI], first posted 2013-03-27; metadata verified via the arXiv API on 2026-10-16T12:24:48.

[24] Shiqing Fan, Xichen Ding, Liang Zhang, and Linjian Mo. Mcptoolbench++: A large scale ai agent model context protocol mcp tool use benchmark, 2025. arXiv:2508.07575 [cs.AI], first posted 2025-08-11; metadata verified via the arXiv API on 2026-10-16T12:24:48.

[25] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks, 2017. arXiv:1706.04599 [cs.LG], first posted 2017-06-14; metadata verified via the arXiv API on 2026-10-16T12:24:48.

[26] Cheng-Yu Hsieh, Yung-Sung Chuang, Chun-Liang Li, Zifeng Wang, Long T. Le, Abhishek Kumar, James Glass, Alexander Ratner, Chen-Yu Lee, Ranjay Krishna, and Tomas Pfister. Found in the middle: Calibrating positional attention bias improves long context utilization, 2024. arXiv:2406.16008 [cs.CL], first posted 2024-06-23; metadata verified via the arXiv API on 2026-10-16T12:24:48.

[27] Wei-Chieh Huang, Weizhi Zhang, Yueqing Liang, Yuanchen Bei, Yankai Chen, Tao Feng, Xinyu Pan, Zhen Tan, Yu Wang, Tianxin Wei, Shanglin Wu, Ruiyao Xu, Liangwei Yang, Rui Yang, Wooseong Yang, Chin-Yuan Yeh, Hanrong Zhang, Haozhen Zhang, Siqi Zhu, Henry Peng Zou, Wanjia Zhao, Song Wang, Wujiang Xu, Zixuan Ke, Zheng Hui, Dawei Li, Yaozu Wu, Langzhou He, Chen Wang, Xiongxiao Xu, Baixiang Huang, Juntao Tan, Shelby Heinecke, Huan Wang, Caiming Xiong, Ahmed A. Metwally, Jun Yan, Chen-Yu Lee, Hanqing Zeng, Yinglong Xia, Xiaokai Wei, Ali Payani, Yu Wang, Haitong Ma, Wenya Wang, Chenguang Wang, Yu Zhang, Xin Eric Wang, Yongfeng Zhang, Jiaxuan You, Hanghang Tong, Xiao Luo, Xue Liu, Yizhou Sun, Wei Wang, Julian McAuley, James Zou, Jiawei Han,

Philip S. Yu, and Kai Shu. A survey of agent memory in the second half: Towards self-evolving and long-horizon agents, 2026. arXiv:2602.06052 [cs.CL], first posted 2026-01-14; metadata verified via the arXiv API on 2026-10-16T12:24:48.

[28] Marcia K. Johnson, Shahin Hashtroudi, and D. Stephen Lindsay. Source monitoring. *Psychological Bulletin*, 114(1):3–28, 1993. DOI metadata verified via the Crossref REST API on 2026-10-16T12:24:48.

[29] John E. Laird. *The Soar Cognitive Architecture*. The MIT Press, 2012. DOI metadata verified via the Crossref REST API on 2026-10-16T12:24:48.

[30] John E. Laird. An analysis and comparison of act-r and soar, 2022. arXiv:2201.09305 [cs.AI], first posted 2022-01-23; metadata verified via the arXiv API on 2026-10-16T12:24:48.

[31] Genglin Liu and Saadia Gabriel. Pm-bench: Evaluating prospective memory in llm agents, 2026. arXiv:2607.12385 [cs.AI], first posted 2026-07-14; metadata verified via the arXiv API on 2026-10-16T12:24:48.

[32] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts, 2023. arXiv:2307.03172 [cs.CL], first posted 2023-07-06; metadata verified via the arXiv API on 2026-10-16T12:24:48.

[33] Zhiling Luo, Xiaorong Shi, Xuanrui Lin, and Jinyang Gao. Evaluation report on mcp servers, 2025. arXiv:2504.11094 [cs.IR], first posted 2025-04-15; metadata verified via the arXiv API on 2026-10-16T12:24:48.

[34] Ziyang Luo, Zhiqi Shen, Wenzhuo Yang, Zirui Zhao, Prathyusha Jwalapuram, Amrita Saha, Doyen Sahoo, Silvio Savarese, Caiming Xiong, and Junnan Li. Mcp-universe: Benchmarking large language models with real-world model context protocol servers, 2025. arXiv:2508.14704 [cs.AI], first posted 2025-08-20; metadata verified via the arXiv API on 2026-10-16T12:24:48.

[35] Mahdi Naser Moghadasi and Faezeh Ghaderi. What twelve llm agent benchmark papers disclose about themselves: A pilot audit and an open scoring schema, 2026. arXiv:2605.21404 [cs.LG], first posted 2026-05-20; metadata verified via the arXiv API on 2026-10-16T12:24:48.

[36] Shiyu Ni, Keping Bi, Lulu Yu, and Jiafeng Guo. Are large language models more honest in their probabilistic or verbalized confidence?, 2024. arXiv:2408.09773 [cs.CL], first posted 2024-08-19; metadata verified via the arXiv API on 2026-10-16T12:24:48.

[37] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems, 2023. arXiv:2310.08560 [cs.AI], first posted 2023-10-12; metadata verified via the arXiv API on 2026-10-16T12:24:48.

[38] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior, 2023. arXiv:2304.03442 [cs.HC], first posted 2023-04-07; metadata verified via the arXiv API on 2026-10-16T12:24:48.

[39] Mohammadreza Rashidi. Unicode tag-block concealment of tool-metadata payloads in the model context protocol: An approval-view fidelity gap across three independent server implementations, 2026. arXiv:2607.05744 [cs.CR], first posted 2026-07-07; metadata verified via the arXiv API on 2026-10-16T12:24:48.

[40] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory, 2025. arXiv:2501.13956 [cs.CL], first posted 2025-01-20; metadata verified via the arXiv API on 2026-10-16T12:24:48.

[41] Jakob Schuster, Vagrant Gautam, and Katja Markert. Whose facts win? llm source preferences under knowledge conflicts, 2026. arXiv:2601.03746 [cs.CL], first posted 2026-01-07; metadata verified via the arXiv API on 2026-10-16T12:24:48.

[42] Minkyu Song. When retrieval fails before it begins: Structurally indirect prerequisite eviction as a retention failure in agentic memory, 2026. arXiv:2608.20400 [cs.AI], first posted 2026-07-05; metadata verified via the arXiv API on 2026-10-16T12:24:48.

[43] Yuru Song and Qi Xin. D-mem: Dopamine-gated agentic memory via reward prediction error routing, 2026. arXiv:2603.14597 [q-bio.NC], first posted 2026-03-15; metadata verified via the arXiv API on 2026-10-16T12:24:48.

[44] Zhaochen Su, Jun Zhang, Xiaoye Qu, Tong Zhu, Yanshu Li, Jiashuo Sun, Juntao Li, Min Zhang, and Yu Cheng. Conflictbank: A benchmark for evaluating the influence of knowledge conflicts in llm, 2024. arXiv:2408.12076 [cs.CL], first posted 2024-08-22; metadata verified via the arXiv API on 2026-10-16T12:24:48.

[45] Balachandra Devarangadi Sunil, Isheeta Sinha, Piyush Maheshwari, Shantanu Todmal, Shreyan Mallik, and Shuchi Mishra. Memory poisoning attack and defense on memory based llm-agents, 2026. arXiv:2601.05504 [cs.CR], first posted 2026-01-09; metadata verified via the arXiv API on 2026-10-16T12:24:48.

[46] Sergei Trashchenkov. Power systems agent benchmark: Executable evaluation of ai agents in electric power engineering, 2026. arXiv:2606.20950 [cs.AI], first posted 2026-06-18; metadata verified via the arXiv API on 2026-10-16T12:24:48.

[47] Endel Tulving and Donald M. Thomson. Encoding specificity and retrieval processes in episodic memory. *Psychological Review*, 80(5):352–373, 1973. DOI metadata verified via the Crossref REST API on 2026-10-16T12:24:48.

[48] Matthew P. Walker and Robert Stickgold. Sleep-dependent learning and memory consolidation. *Neuron*, 44(1):121–133, 2004. DOI metadata verified via the Crossref REST API on 2026-10-16T12:24:48.

[49] Qihan Wang, Nicholas Tomlin, Michael Hu, Brian Dillon, and Tal Linzen. Simulating human memory with language models, 2026. arXiv:2605.25680 [cs.CL], first posted 2026-05-25; metadata verified via the arXiv API on 2026-10-16T12:24:48.

[50] Zi Wang, Xingqiao Wang, Emmanuel Addai, Devika Ambekar, and Xiaowei Xu. The right memory in the wrong context: Verifying retrieval admissibility in long-term agent memory, 2026. arXiv:2610.07309 [cs.AI], first posted 2026-10-05; metadata verified via the arXiv API on 2026-10-16T12:24:48.

[51] Miao Xiong, Zhiyuan Hu, Xinyang Lu, Yifei Li, Jie Fu, Junxian He, and Bryan Hooi. Can llms express their uncertainty? an empirical evaluation of confidence elicitation in llms, 2023. arXiv:2306.13063 [cs.CL], first posted 2023-06-22; metadata verified via the arXiv API on 2026-10-16T12:24:48.

[52] Peng Xu, Zuyu Zhang, Yuze Sun, Feng Tian, Long Wang, and Chen Zhang. Contextpipe: Database-inspired context assembly for long-horizon agents, 2026. arXiv:2609.00749 [cs.AI], first posted 2026-09-01; metadata verified via the arXiv API on 2026-10-16T12:24:48.

[53] Rongwu Xu, Zehan Qi, Zhijiang Guo, Cunxiang Wang, Hongru Wang, Yue Zhang, and Wei Xu. Knowledge conflicts for llms: A survey, 2024. arXiv:2403.08319 [cs.CL], first posted 2024-03-13; metadata verified via the arXiv API on 2026-10-16T12:24:48.

[54] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-mem: Agentic memory for llm agents, 2025. arXiv:2502.12110 [cs.CL], first posted 2025-02-17; metadata verified via the arXiv API on 2026-10-16T12:24:48.

[55] Zhentao Xu, Shangjin Zhang, Emir Poyraz, Yvonne Li, Ye Jin, Xie Lu, Xiaoyang Gu, Karthik Ramgopal, Praveen Kumar Bodigutla, and Xiaofeng Wang. Hierarchical long-term semantic memory for linkedin's hiring agent, 2026. arXiv:2604.26197 [cs.IR], first posted 2026-04-29; metadata verified via the arXiv API on 2026-10-16T12:24:48.

[56] Daniel Yang, Yao-Hung Hubert Tsai, and Makoto Yamada. On verbalized confidence scores for llms, 2024. arXiv:2412.14737 [cs.CL], first posted 2024-12-19; metadata verified via the arXiv API on 2026-10-16T12:24:48.

[57] Zeyu Zhang, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Quanyu Dai, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. A survey on the memory mechanism of large language model based agents, 2024. arXiv:2404.13501 [cs.AI], first posted 2024-04-21; metadata verified via the arXiv API on 2026-10-16T12:24:48.

[58] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. Memorybank: Enhancing large language models with long-term memory, 2023. arXiv:2305.10250 [cs.CL], first posted 2023-05-17; metadata verified via the arXiv API on 2026-10-16T12:24:48.

[59] Chenyu Zhou, Huacan Chai, Wenteng Chen, Zihan Guo, Rong Shan, Yuanyi Song, Tianyi Xu, Yingxuan Yang, Aofan Yu, Weiming Zhang, Congming Zheng, Jiachen Zhu, Zeyu Zheng, Zhuosheng Zhang, Xingyu Lou, Changwang Zhang, Zhihui Fu, Jun Wang, Weiwen Liu, Jianghao Lin, and Weinan Zhang. Externalization in llm agents: A unified review of memory, skills, protocols and harness engineering, 2026. arXiv:2604.08224 [cs.SE], first posted 2026-04-09; metadata verified via the arXiv API on 2026-10-16T12:24:48.

[60] Wei Zhou, Xuanhe Zhou, Shaokun Han, Hongming Xu, Guoliang Li, Zhiyu Li, Feiyu Xiong, and Fan Wu. Are we ready for an agent-native memory system?, 2026. arXiv:2606.24775 [cs.CL], first posted 2026-06-23; metadata verified via the arXiv API on 2026-10-16T12:24:48.

[61] Xuanjun Zong, Zhiqi Shen, Lei Wang, Yunshi Lan, and Chao Yang. Mcp-safetybench: A benchmark for safety evaluation of large language models with real-world mcp servers, 2025. arXiv:2512.15163 [cs.CL], first posted 2025-12-17; metadata verified via the arXiv API on 2026-10-16T12:24:48.